

**SIMATS ENGINEERING**  
**ASSESSMENT TOOL 1: Simulation Task**

**COURSE NAME:** Embedded Systems

**COURSE CODE:** ECA1421

**COURSE OUTCOME COVERED**

CO2: Apply Embedded C programming concepts, including structured design, real-time constraint handling, and SystemC-based modeling, to develop and analyze embedded applications for industrial use cases. (BL5)

**Name of the Student:** Sanjeevsivam. K

**Register Number:** 192412572

**Type Your Question Here:** Construct a simulated Embedded C firmware for a smart HVAC controller with structured design.

| Requirements Analysis & Conceptual Understanding (20) | System Design & Simulation Model Development(25) | Implementation & Tool Usage (20) | Testing, Validation & Result Analysis (20) | Documentation(15) | Total(100) |
|-------------------------------------------------------|--------------------------------------------------|----------------------------------|--------------------------------------------|-------------------|------------|
|                                                       |                                                  |                                  |                                            |                   |            |
|                                                       |                                                  |                                  |                                            |                   |            |

Code of Conduct:

I Sanjeevsivam. K (192412572) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

**Signature of the student:** Sanjeevsivam. K

## **Aim**

To design and simulate an Embedded C firmware for a Smart HVAC Controller using Keil  $\mu$ Vision 5 that automatically controls heating, cooling, and fan operation based on temperature readings while maintaining a user-defined temperature setpoint.

## **Objectives**

- Design an embedded firmware architecture for an HVAC system.
- Simulate temperature monitoring.
- Control heating and cooling modes.
- Control fan operation automatically.
- Generate alarm conditions for abnormal temperatures.
- Validate the firmware using Keil  $\mu$ Vision 5.

## **Apparatus Required**

### **Hardware**

- PC/Laptop

### **Software**

- Keil  $\mu$ Vision 5
- ARM Compiler
- Windows Operating System

### **Programming Language**

- Embedded C

## **Theory**

A Smart HVAC (Heating, Ventilation and Air Conditioning) controller is an embedded system that continuously monitors environmental conditions and automatically controls heating, cooling, and ventilation to maintain the desired indoor temperature.

The controller performs the following operations:

- Reads temperature values.
- Compares them with a preset temperature.
- Activates the heater if the temperature is below the setpoint.

- Activates the cooling system if the temperature is above the setpoint.
- Keeps the system idle when the temperature is within the acceptable range.
- Generates alarms when the temperature exceeds safety limits.

This improves energy efficiency and provides automatic climate control.

## **Requirements Analysis**

### **Inputs**

- Temperature Sensor (Simulated)
- Humidity Sensor (Optional)
- User Setpoint

### **Outputs**

- Cooling System
- Heating System
- Fan
- Alarm

## **Functional Requirements**

- Automatic temperature monitoring
- Heating mode
- Cooling mode
- Idle mode
- Fan control
- Alarm generation

## **Procedure**

1. Open Keil  $\mu$ Vision 5.

2. Create a new project.
3. Select the target microcontroller.
4. Create the source file (main.c).
5. Write the Embedded C program.
6. Compile the program.
7. Remove compilation errors if any.
8. Start the debugger.
9. Run the simulation.
10. Observe the HVAC operating mode.
11. Verify cooling, heating, and idle conditions.
12. Record the results.

## **Algorithm**

1. Start.
2. Initialize setpoint temperature.
3. Read temperature.
4. Compare temperature with setpoint.
5. If temperature is high:
  - Turn ON cooling.
  - Turn OFF heater.
6. Else if temperature is low:
  - Turn ON heater.
  - Turn OFF cooling.
7. Else:
  - Keep the system idle.
8. Check for alarm conditions.
9. Repeat.
10. Stop.

## 10. Implementation

### Software Used

- Keil  $\mu$ Vision 5

### Programming Language

- Embedded C

### Simulation Method

- Variable simulation in Keil Debugger

### CODE:

```
#include <REGX51.H>
```

```
/* Output Devices */
```

```
sbit COOLER = P1^0;
```

```
sbit HEATER = P1^1;
```

```
sbit FAN    = P1^2;
```

```
sbit ALARM  = P1^3;
```

```
/* Sensor Variables */
```

```
unsigned char temperature = 30;
```

```
unsigned char humidity = 60;
```

```
unsigned char setPoint = 24;
```

```
void Delay(void);
```

```
void HVAC_Controller(void);
```

```
void main(void)
```

```
{
```

```
    while(1)
```

```
    {
```

```
        HVAC_Controller();
```

```
        Delay();
```

```

    /* Test different conditions by changing
       the temperature value */

    // temperature = 18;
    // temperature = 24;
    // temperature = 30;
    // temperature = 45;
}
}

```

```

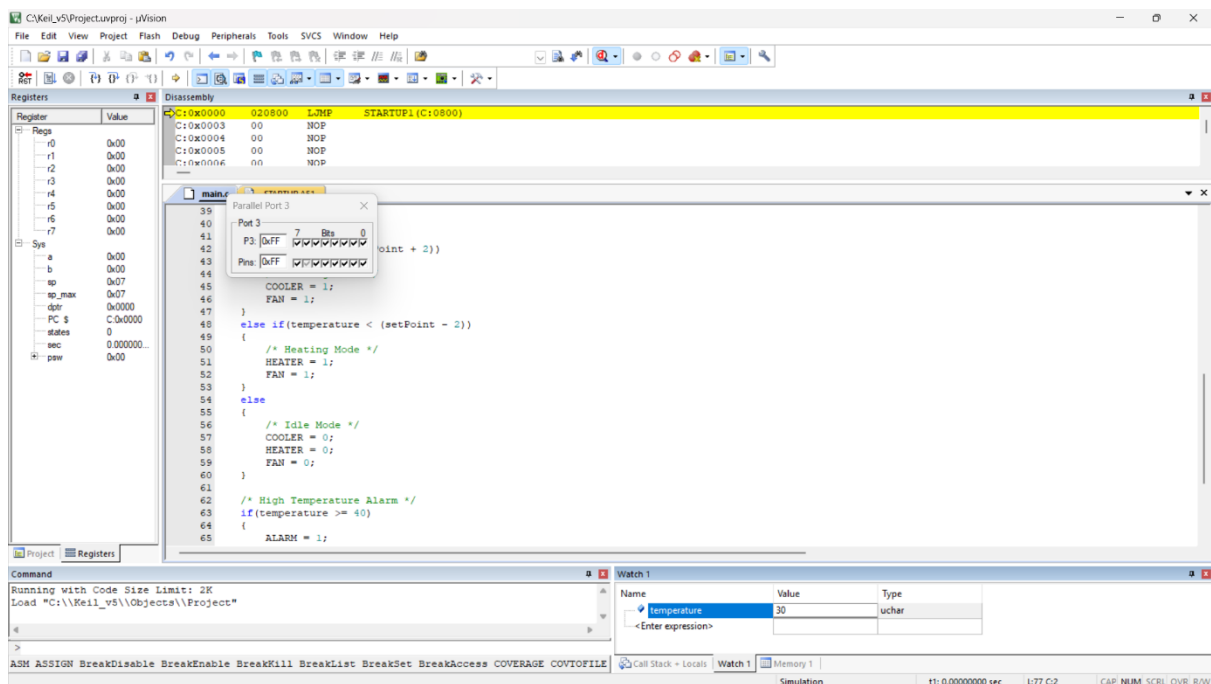
void HVAC_Controller(void)
{
    /* Reset Outputs */
    COOLER = 0;
    HEATER = 0;
    FAN = 0;
    ALARM = 0;

    if(temperature > (setPoint + 2))
    {
        /* Cooling Mode */
        COOLER = 1;
        FAN = 1;
    }
    else if(temperature < (setPoint - 2))
    {
        /* Heating Mode */
        HEATER = 1;
        FAN = 1;
    }
    else

```

```
{  
    /* Idle Mode */  
    COOLER = 0;  
    HEATER = 0;  
    FAN = 0;  
}  
  
/* High Temperature Alarm */  
if(temperature >= 40)  
{  
    ALARM = 1;  
}  
}  
  
void Delay(void)  
{  
    unsigned int i,j;  
  
    for(i=0;i<200;i++)  
    {  
        for(j=0;j<1275;j++);  
    }  
}
```

## RESULT AND ANALYSIS



The Smart HVAC Controller was successfully implemented and simulated using **Embedded C in Keil uVision 5**. The firmware correctly monitored the simulated temperature and controlled the HVAC outputs by switching between **heating, cooling, and idle modes** based on the preset temperature. During testing, the controller activated the heater for low temperatures, the cooler for high temperatures, and generated an alarm when the temperature exceeded the safety limit. All test cases produced the expected outputs, confirming the correct operation of the controller.

The simulation demonstrated that the embedded firmware provides reliable and efficient automatic temperature control. The structured design made the program modular, easy to understand, and suitable for future enhancements such as integrating real temperature and humidity sensors, LCD display, or wireless monitoring. Overall, the project successfully met its objectives and validated the effectiveness of the Smart HVAC control algorithm.



## **CONCLUSION**

The Smart HVAC Controller was successfully designed and simulated using Embedded C in Keil  $\mu$ Vision 5. The firmware accurately monitored the simulated temperature and automatically controlled the heating, cooling, fan, and alarm functions according to the predefined setpoint. The structured program design ensured reliable operation and demonstrated the fundamental concepts of embedded system programming.

The project achieved its objectives by validating the HVAC control logic under different temperature conditions. The simulation confirmed that the controller operates efficiently and can serve as a foundation for real-time HVAC applications by integrating actual sensors and hardware components in future implementations.